Calendar

IT 103 – Computer Programming 2

Medina, Dustin Paul L.

Corrales, Carl

April 15, 2026

System Description

Overview

An appointment system is a digital platform that allows users to schedule and manage their appointments, such as clinic visits, meetings, or personal events. It helps users stay organized by providing a simple, user-friendly interface to add, edit, and cancel appointments anytime.

In addition, the system can organize appointments by date, time, or category, making it easier to view and manage daily or weekly schedules. To improve efficiency, the system may also detect schedule conflicts, preventing overlapping appointments. All data can be saved, allowing users to access their appointments even after closing the program. Overall, the appointment system is designed to make scheduling easier, more organized, and more reliable for everyday use.

Target Users

This system is designed for the intent of usage for students. professionals, and anyone seeking a clean and easy to use appointment system. It is particularly useful for individuals who need to manage their appointments, check for any conflicting appointments, and without relying on an internet connection.

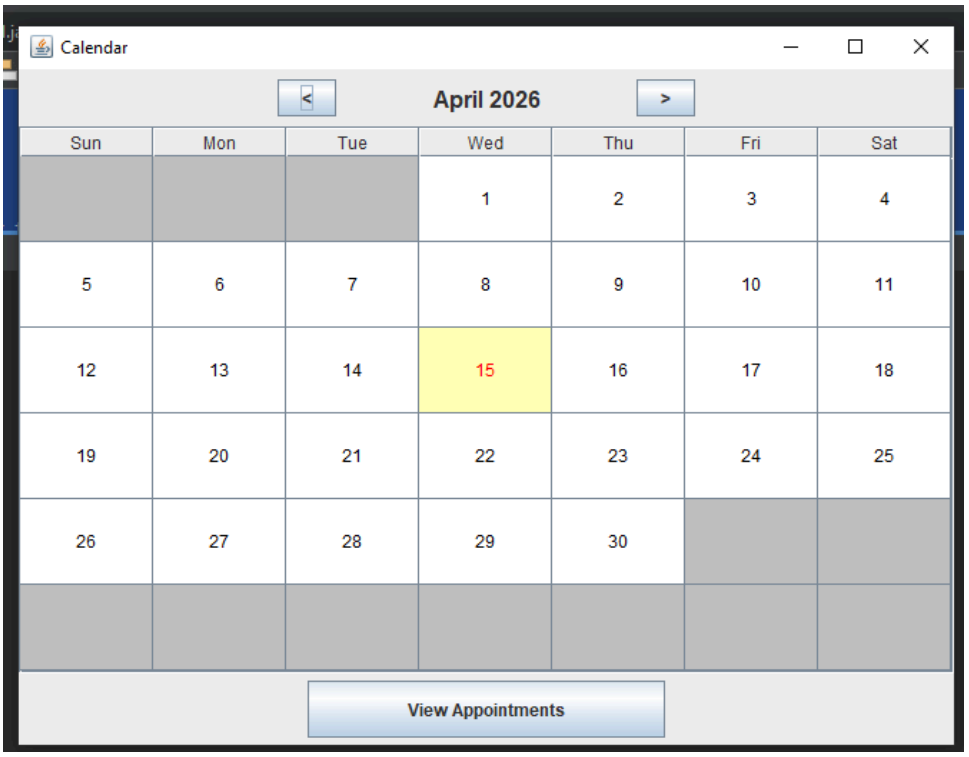
Features:

- **Interactive Calendar GUI -** Program displays a calendar with the similar look and feel of its real world counterpart, giving the user a sense of familiarity when booting up for the first time.
- **Appointment Manager -** User is given the ability to add, view, and edit appointments at ease with just a few clicks.
- **Time Conflict Detection -** Program automatically detects appointments for time entries set by the user avoiding overlappings in the process.
- **Status Tracking -** Simple text indicators to tell the user if their appointments are pending or not.

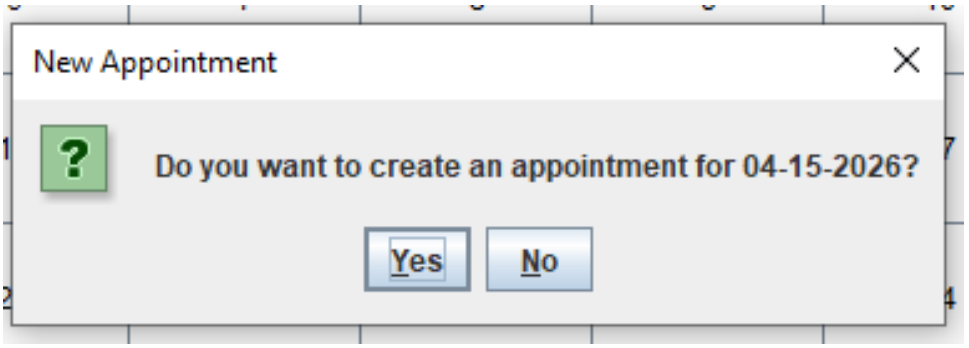
- **Built-in Save System -** Program has a built-in save system set to a local text file once the user closes the program ensuring the progress is saved even without an internet connection.

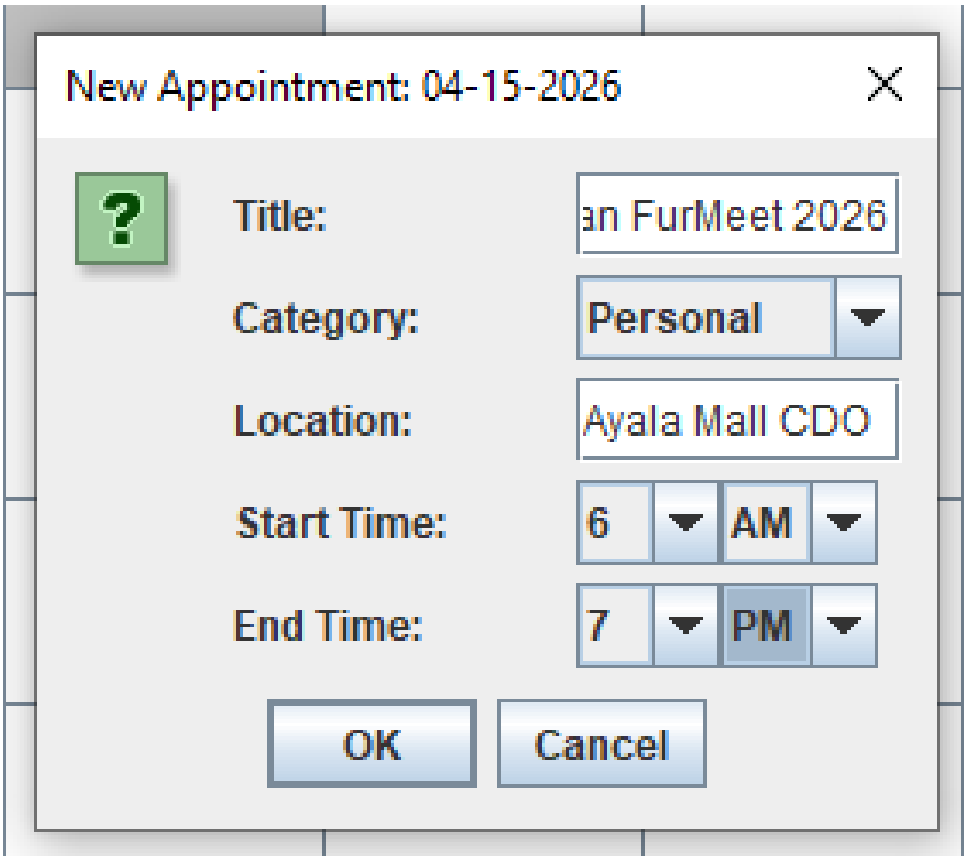
How to Use

1. Open the application. Use the **"<"** and **">"** buttons at the top to navigate between different months and years of your desired appointment.

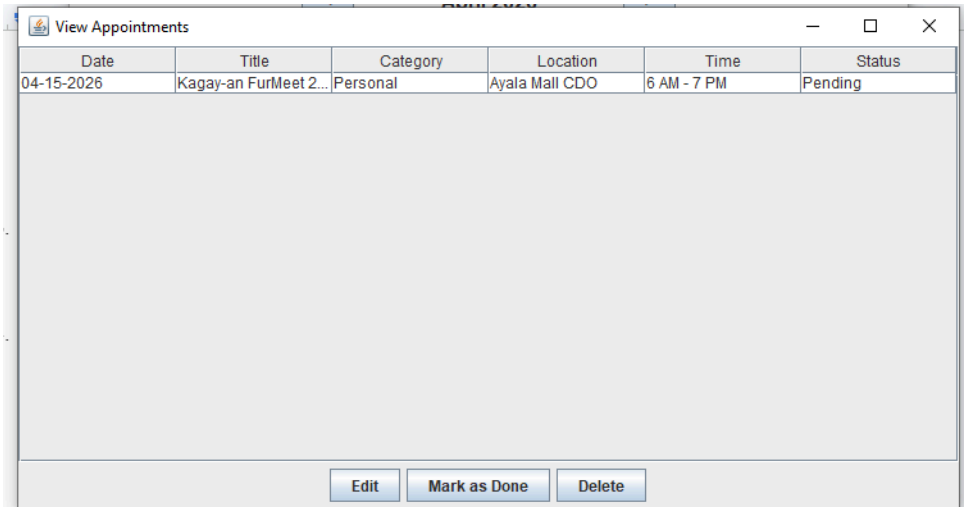


2. Click on any specific day in the calendar GUI. A prompt will ask if you wish to create an appointment. Click **"Yes"** if you want and fill out the Title, Category, Location, and Times. Click **"OK"** then afterwards.

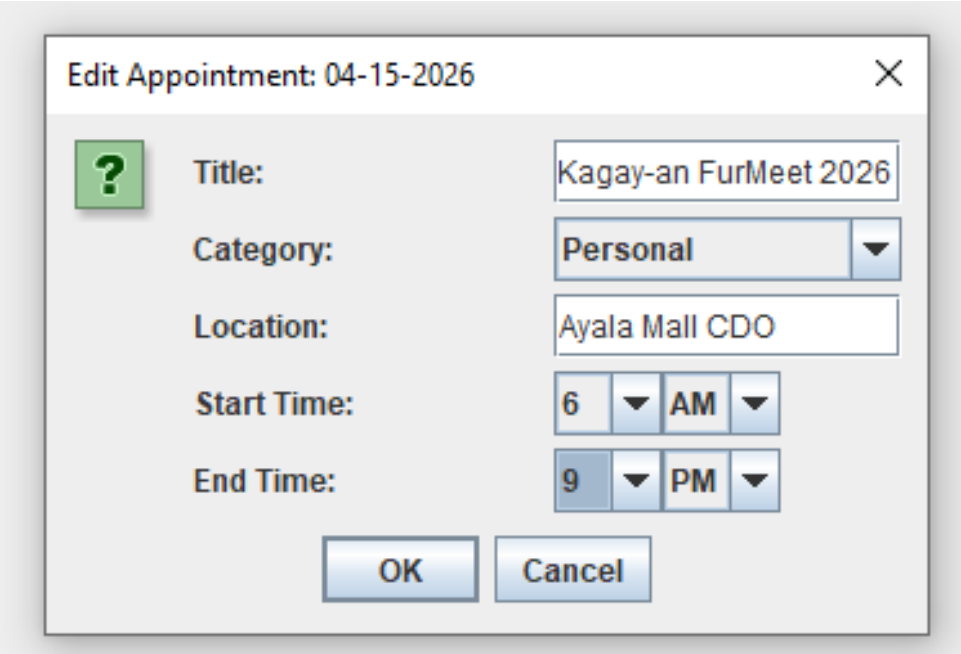


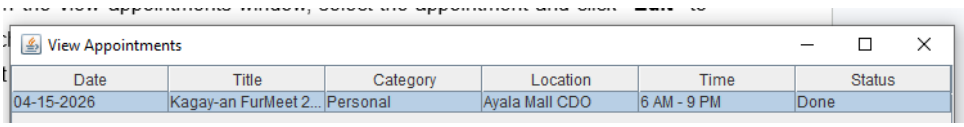


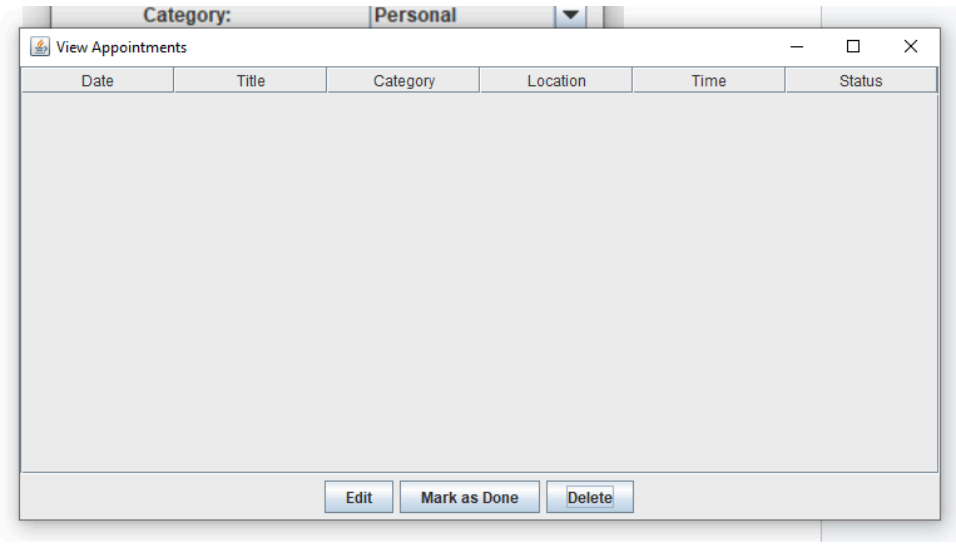
3. Click the "**View Appointments"** button at the bottom of the screen. A new window will appear listing all your appointments.



4. In the view appointments window, select the appointment and click **“Edit”** to change its details, “**Mark as Done”** to update its status, or “**Delete”** to remove it entirely. Note that once you mark it as done, there's no going back.







[Flowchart]

Source Code

CalendarGUI.java

```
// JavaCalendarApplication.java is used for the defence.
package lepackage;

// Import necessary libraries for GUI, events, file I/O, and data structures
import javax.swing.*;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.util.*;
import java.util.List;

/**
 * Main GUI class for the Calendar application.
 * Acts as the primary window and holds the shared data structure.
 */
public class CalendarGUI extends JFrame {
    // UI Components for the main window
    private JLabel monthLabel;
    private DefaultTableModel calendarModel;
    private JTable calendarTable;

    // State variables to track the currently viewed month and year
    private int currentMonth;
    private int currentYear;
```

```
    // Variables to store the actual, real-world current date
    private int realDay;
    private int realMonth;
    private int realYear;

    // Shared data structure for appointments. Key: Date String, Value: List of
Appointments.
    // Accessed by the other classes via getters.
    private Map<String, List<Appointment>> appointments = new HashMap<>();

    // File name for persistent data storage
    private final String FILE_NAME = "calendarsaveddata.txt";

    /**
     * Constructor: Initializes the main frame, loads data, and builds the UI.
     */
    public CalendarGUI() {
        // Set up the main application window
        setTitle("Calendar");
        setSize(670, 512);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE); // Custom
logic handles saving on exit
        setLayout(new BorderLayout()); // Top, Center, Bottom layout

        // Initialize background data and load saved appointments
        initializeRealDate();
        loadAppointments();

        // Build the three main sections of the UI
        initializeTopNavigation();
        initializeCalendarTable();
        initializeBottomPanel();

        // Add listener to save data when the window is closed
```

```
        initializeWindowListener();

        // Populate the initial calendar view
        updateCalendarDisplay();
    }

    /**
     * Captures today's exact date to highlight it and set the initial view.
     */
    private void initializeRealDate() {
        GregorianCalendar calendar = new GregorianCalendar();
        realDay = calendar.get(Calendar.DAY_OF_MONTH);
        realMonth = calendar.get(Calendar.MONTH);
        realYear = calendar.get(Calendar.YEAR);

        // Default the viewing calendar to the current real month
        currentMonth = realMonth;
        currentYear = realYear;
    }

    /**
     * Creates the top panel containing the Previous/Next buttons and the Month/Year
label.
     */
    private void initializeTopNavigation() {
        JPanel topPanel = new JPanel();
        JButton previousButton = new JButton("<");
        JButton nextButton = new JButton(">");

        // Setup the label that displays "Month Year"
        monthLabel = new JLabel("", SwingConstants.CENTER);
        monthLabel.setFont(new Font("SansSerif", Font.BOLD, 16));
        monthLabel.setPreferredSize(new Dimension(200, 30));
```

```
        // Attach actions to change the month
        previousButton.addActionListener(event -> changeMonth(-1));
        nextButton.addActionListener(event -> changeMonth(1));

        topPanel.add(previousButton);
        topPanel.add(monthLabel);
        topPanel.add(nextButton);

        // Add to the top of the main layout
        add(topPanel, BorderLayout.NORTH);
    }

    /**
     * Sets up the JTable that acts as the visual calendar grid.
     */
    private void initializeCalendarTable() {
        String[] daysOfWeek = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};

        // Create a table model that prevents direct typing in cells
        calendarModel = new DefaultTableModel(null, daysOfWeek) {
            public boolean isCellEditable(int row, int column) {
                return false;
            }
        };

        calendarTable = new JTable(calendarModel);
        calendarTable.setRowHeight(60); // Make cells large enough to look like
calendar days

        // Apply custom renderer to colorize specific cells (e.g., today, appointments)
        calendarTable.setDefaultRenderer(Object.class, new CalendarRenderer());

        // Listen for mouse clicks to add appointments to specific days
        calendarTable.addMouseListener(new MouseAdapter() {
```

```
            public void mouseClicked(MouseEvent event) {
                int row = calendarTable.rowAtPoint(event.getPoint());
                int column = calendarTable.columnAtPoint(event.getPoint());
                Object cellValue = calendarModel.getValueAt(row, column);

                // If the user clicked on a valid day number, prompt to add an appointment
                if (cellValue != null) {
                    handleDateSelection((Integer) cellValue);
                }
            }
        });

        add(new JScrollPane(calendarTable), BorderLayout.CENTER);
    }

    /**
     * Creates the bottom panel containing the "View Appointments" button.
     */
    private void initializeBottomPanel() {
        JPanel bottomPanel = new JPanel();
        JButton manageButton = new JButton("View Appointments");
        manageButton.setPreferredSize(new Dimension(250, 40));

        // Opens the secondary view via the separated ViewAppointment class
        manageButton.addActionListener(event -> new
ViewAppointment(this).setVisible(true));

        bottomPanel.add(manageButton);
        add(bottomPanel, BorderLayout.SOUTH);
    }

    /**
     * Intercepts the window closing event to save data before terminating.
     */
```

```
    private void initializeWindowListener() {
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent event) {
                saveAppointments(); // Save memory to text file
                dispose();          // Close window resources
                System.exit(0);     // End process
            }
        });
    }

    /**
     * Triggered when a day is clicked. Initiates the AddAppointment logic.
     */
    private void handleDateSelection(int day) {
        int monthIndex = currentMonth + 1; // Calendar months are 0-indexed

        // Format date string to mm-dd-yyyy
        String dateKey = String.format("%02d-%02d-%04d", monthIndex, day,
currentYear);

        int userChoice = JOptionPane.showConfirmDialog(
            this,
            "Do you want to create an appointment for " + dateKey + "?",
            "New Appointment",
            JOptionPane.YES_NO_OPTION,
            JOptionPane.QUESTION_MESSAGE
        );

        if (userChoice == JOptionPane.YES_OPTION) {
            // Delegates the input form logic to the separated AddAppointment class
            AddAppointment addDialog = new AddAppointment(this, dateKey, null);
            addDialog.showDialog();
        }
```

```
}

/**
 * Saves the appointment map to a text file using a pipe-delimited format.
 */
private void saveAppointments() {
    try (PrintWriter writer = new PrintWriter(new FileWriter(FILE_NAME))) {
        for (String dateKey : appointments.keySet()) {
            for (Appointment appointment : appointments.get(dateKey)) {
                writer.println(
                    dateKey + "|" +
                    appointment.title + "|" +
                    appointment.startTime + "|" +
                    appointment.endTime + "|" +
                    appointment.location + "|" +
                    appointment.category + "|" +
                    appointment.status
                );
            }
        }
    } catch (IOException exception) {
        exception.printStackTrace();
    }
}

/**
 * Loads appointment data from the text file into the map on startup.
 */
private void loadAppointments() {
    File file = new File(FILE_NAME);
    if (!file.exists()) return; // Skip if no previous saves exist

    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
```

```
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split("\\|");

                // Ensure the line has enough parts before parsing
                if (parts.length >= 7) {
                    // Backwards compatibility for older save versions
                    String category = parts.length >= 8 ? parts[6] : parts[5];
                    String status = parts.length >= 8 ? parts[7] : parts[6];

                    Appointment appointment = new Appointment(parts[1], parts[2], parts[3],
parts[4], category);
                    appointment.status = status;

                    // Add to map, creating the list if it doesn't exist for this date
                    appointments.computeIfAbsent(parts[0], key -> new
ArrayList<>()).add(appointment);
                }
            }
        } catch (Exception exception) {
            exception.printStackTrace();
        }
    }

    /**
     * Adjusts the currently viewed month and handles year rollovers.
     */
    private void changeMonth(int offset) {
        currentMonth += offset;

        if (currentMonth < 0) {
            currentMonth = 11; // Wrap back to December
            currentYear--;     // Decrement year
        } else if (currentMonth > 11) {
            currentMonth = 0;  // Wrap forward to January
```

```
            currentYear++;     // Increment year
        }

        updateCalendarDisplay(); // Refresh grid
    }

    /**
     * Calculates the days of the selected month and populates the JTable grid.
     * Made public so external classes can trigger visual updates.
     */
    public void updateCalendarDisplay() {
        String[] monthNames = {
            "January", "February", "March", "April", "May", "June",
            "July", "August", "September", "October", "November", "December"
        };

        // Update top label
        monthLabel.setText(monthNames[currentMonth] + " " + currentYear);

        // Reset table rows
        calendarModel.setRowCount(0);
        calendarModel.setRowCount(6); // Max 6 rows needed for a month

        // Determine the starting day of the week for the 1st of the month
        GregorianCalendar calendar = new GregorianCalendar(currentYear,
currentMonth, 1);
        int startDayOfWeek = calendar.get(Calendar.DAY_OF_WEEK) - 1;
        int totalDaysInMonth =
calendar.getActualMaximum(Calendar.DAY_OF_MONTH);

        int row = 0;
        int column = startDayOfWeek;

        // Fill the grid sequentially
```

```
        for (int dayOfMonth = 1; dayOfMonth <= totalDaysInMonth; dayOfMonth++) {
            calendarModel.setValueAt(dayOfMonth, row, column);
            column++;

            // Move to the next row when reaching Sunday
            if (column > 6) {
                column = 0;
                row++;
            }
        }
    }

    /**
     * Public getter allowing other classes to read/modify the appointment map.
     */
    public Map<String, List<Appointment>> getAppointments() {
        return appointments;
    }

    /**
     * Custom TableCellRenderer to handle background colors of the calendar cells.
     */
    class CalendarRenderer extends DefaultTableCellRenderer {
        @Override
        public Component getTableCellRendererComponent(JTable table, Object value,
boolean isSelected, boolean hasFocus, int row, int column) {
            Component component = super.getTableCellRendererComponent(table,
value, isSelected, hasFocus, row, column);

            if (value != null) {
                int day = (Integer) value;
                int monthIndex = currentMonth + 1;
                String dateKey = String.format("%02d-%02d-%04d", monthIndex, day,
currentYear);
```

```
            boolean isToday = (currentYear == realYear && currentMonth ==
realMonth && day == realDay);
            boolean hasAppointments = appointments.containsKey(dateKey);

            // Color Logic
            if (isToday) {
                setBackground(new Color(255, 255, 180)); // Yellow for current real day
                component.setForeground(Color.RED);
            } else if (hasAppointments) {
                // Check if all appointments on this day are done
                boolean allTasksDone =
appointments.get(dateKey).stream().allMatch(appointment ->
appointment.status.equalsIgnoreCase("DONE"));

                // Green if all done, Blue if pending tasks exist
                setBackground(allTasksDone ? new Color(180, 255, 180) : new
Color(180, 220, 255));
                component.setForeground(Color.BLACK);
            } else {
                setBackground(Color.WHITE); // Default empty day
                component.setForeground(Color.BLACK);
            }
        } else {
            setBackground(Color.LIGHT_GRAY); // Blank cells outside the month
        }

        setHorizontalAlignment(CENTER);
        return component;
    }
}

/**
 * Data object representing a single appointment.
```

```
     * Public static so external classes can instantiate and utilize it.
     */
    public static class Appointment {
        public String title;
        public String startTime;
        public String endTime;
        public String location;
        public String category;
        public String status;

        public Appointment(String title, String startTime, String endTime, String location,
String category) {
            this.title = title;
            this.startTime = startTime;
            this.endTime = endTime;
            this.location = location;
            this.category = category;
            this.status = "Pending"; // Always default to pending upon creation
        }
    }

    /**
     * Application entry point.
     */
    public static void main(String[] args) {
        // Ensure thread safety for Swing components
        SwingUtilities.invokeLater(() -> new CalendarGUI().setVisible(true));
    }
}
```

AddAppointment.java

```
package lepackage;

import javax.swing.*;
```

```
import java.awt.*;
import java.util.ArrayList;

/**
 * Controller class to handle the logic and UI dialogs for adding or editing an
appointment.
 * Centralizes data validation and conflict checking.
 */
public class AddAppointment {
    private CalendarGUI mainGUI;     // Needed to access the global map and trigger
UI updates
    private String dateKey;          // The specific date being modified
    private CalendarGUI.Appointment existingAppt; // Holds data if we are editing, null
if creating new

    /**
     * Constructor
     * @param mainGUI Reference to main program
     * @param dateKey The string ID (mm-dd-yyyy) of the date
     * @param existingAppt If null, acts as "Create". If provided, acts as "Edit".
     */
    public AddAppointment(CalendarGUI mainGUI, String dateKey,
CalendarGUI.Appointment existingAppt) {
        this.mainGUI = mainGUI;
        this.dateKey = dateKey;
        this.existingAppt = existingAppt;
    }

    /**
     * Builds and shows the input dialog. Processes the user's input.
     * @return true if an appointment was successfully saved/updated, false if
canceled or failed validation.
     */
    public boolean showDialog() {
```

```
        // Initialize UI input fields. Pre-fill with existingAppt data if in Edit mode.
        JTextField titleField = new JTextField(existingAppt != null ? existingAppt.title :
"");
        JTextField locationField = new JTextField(existingAppt != null ?
existingAppt.location : "");
        JComboBox<String> categoryBox = new JComboBox<>(new String[]{"School",
"Work", "Personal"});
        if (existingAppt != null) {
            categoryBox.setSelectedItem(existingAppt.category);
        }

        // Time selection dropdown arrays
        String[] hours = {"1","2","3","4","5","6","7","8","9","10","11","12"};
        String[] ampmOptions = {"AM", "PM"};

        // Start Time components
        JComboBox<String> startHourBox = new JComboBox<>(hours);
        JComboBox<String> startAmPmBox = new JComboBox<>(ampmOptions);

        // End Time components
        JComboBox<String> endHourBox = new JComboBox<>(hours);
        JComboBox<String> endAmPmBox = new JComboBox<>(ampmOptions);

        // Pre-fill time dropdowns if in Edit mode
        if (existingAppt != null) {
            String[] startParts = existingAppt.startTime.split(" ");
            startHourBox.setSelectedItem(startParts[0]);
            startAmPmBox.setSelectedItem(startParts[1]);

            String[] endParts = existingAppt.endTime.split(" ");
            endHourBox.setSelectedItem(endParts[0]);
            endAmPmBox.setSelectedItem(endParts[1]);
        }
```

```
        // Layout the fields into a panel
        JPanel inputPanel = buildAppointmentInputPanel(
            titleField, categoryBox, locationField,
            startHourBox, startAmPmBox, endHourBox, endAmPmBox
        );

        // Dynamically set title based on mode
        String dialogTitle = existingAppt == null ? "New Appointment: " + dateKey : "Edit
Appointment: " + dateKey;

        // Show dialog and wait for user response
        int result = JOptionPane.showConfirmDialog(mainGUI, inputPanel, dialogTitle,
JOptionPane.OK_CANCEL_OPTION);

        // Process data if user clicked OK
        if (result == JOptionPane.OK_OPTION) {

            // Validation 1: Title cannot be blank
            if (titleField.getText().trim().isEmpty()) {
                JOptionPane.showMessageDialog(mainGUI, "The event title cannot be
blank.", "Warning", JOptionPane.WARNING_MESSAGE);
                return false;
            }

            // Construct full time strings for storage
            String fullStartTime = startHourBox.getSelectedItem() + " " +
startAmPmBox.getSelectedItem();
            String fullEndTime = endHourBox.getSelectedItem() + " " +
endAmPmBox.getSelectedItem();

            // Convert times to integers (minutes since midnight) for mathematical
comparisons
            int startInMinutes = convertToMinutes((String)
startHourBox.getSelectedItem(), (String) startAmPmBox.getSelectedItem());
```

```
        int endInMinutes = convertToMinutes((String) endHourBox.getSelectedItem(),
(String) endAmPmBox.getSelectedItem());

        // Validation 2: Logic check (cannot end before starting)
        if (endInMinutes <= startInMinutes) {
            JOptionPane.showMessageDialog(mainGUI, "End time must be after start
time!", "Error", JOptionPane.ERROR_MESSAGE);
            return false;
        }

        // Validation 3: Prevent overlapping appointments
        if (hasTimeConflict(dateKey, startInMinutes, endInMinutes, existingAppt)) {
            return false;
        }

        // If we are editing, remove the old version before creating the new one
        if (existingAppt != null) {
            mainGUI.getAppointments().get(dateKey).remove(existingAppt);
        }

        // Create the new data object
        CalendarGUI.Appointment newAppointment = new
CalendarGUI.Appointment(
            titleField.getText().trim(), fullStartTime, fullEndTime,
            locationField.getText().trim(), (String) categoryBox.getSelectedItem()
        );

        // Preserve the original status if editing (so we don't accidentally reset "Done"
to "Pending")
        if (existingAppt != null) {
            newAppointment.status = existingAppt.status;
        }

        // Insert into the main data structure
```

```
            mainGUI.getAppointments().computeIfAbsent(dateKey, key -> new
ArrayList<>()).add(newAppointment);

            // Tell the main GUI to refresh the calendar grid colors
            mainGUI.updateCalendarDisplay();
            return true;
        }
        return false;
    }

    /**
     * Helper method to organize the layout of the input fields.
     */
    private JPanel buildAppointmentInputPanel(
        JTextField titleField, JComboBox<String> categoryBox, JTextField locationField,
        JComboBox<String> startHourBox, JComboBox<String> startAmPmBox,
        JComboBox<String> endHourBox, JComboBox<String> endAmPmBox) {

        JPanel panel = new JPanel(new GridLayout(0, 2, 5, 5)); // 2 columns

        // Group start time elements
        JPanel startPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
        startPanel.add(startHourBox);
        startPanel.add(startAmPmBox);

        // Group end time elements
        JPanel endPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
        endPanel.add(endHourBox);
        endPanel.add(endAmPmBox);

        // Add pairs of Label -> Field
        panel.add(new JLabel("Title:"));
        panel.add(titleField);
        panel.add(new JLabel("Category:"));
```

```
        panel.add(categoryBox);
        panel.add(new JLabel("Location:"));
        panel.add(locationField);
        panel.add(new JLabel("Start Time:"));
        panel.add(startPanel);
        panel.add(new JLabel("End Time:"));
        panel.add(endPanel);

        return panel;
    }

    /**
     * Checks if the newly entered time range overlaps with any existing appointments
on the same day.
     */
    private boolean hasTimeConflict(String dateKey, int newStartInMinutes, int
newEndInMinutes, CalendarGUI.Appointment ignoreAppt) {
        // No conflict possible if there are no other appointments today
        if (!mainGUI.getAppointments().containsKey(dateKey)) {
            return false;
        }

        for (CalendarGUI.Appointment existingAppt :
mainGUI.getAppointments().get(dateKey)) {
            // Skip checking the appointment against itself when editing
            if (existingAppt == ignoreAppt) continue;

            String[] startParts = existingAppt.startTime.split(" ");
            String[] endParts = existingAppt.endTime.split(" ");

            int existingStart = convertToMinutes(startParts[0], startParts[1]);
            int existingEnd = convertToMinutes(endParts[0], endParts[1]);

            // Overlap logic: New starts before old ends, AND new ends after old starts
```

```
            if (newStartInMinutes < existingEnd && newEndInMinutes > existingStart) {
                String conflictMessage = "Conflict with '" + existingAppt.title +
                                        "' (" + existingAppt.startTime + " - " + existingAppt.endTime
+ ")";
                JOptionPane.showMessageDialog(mainGUI, conflictMessage, "Error",
JOptionPane.ERROR_MESSAGE);
                return true;
            }
        }
        return false;
    }

    /**
     * Converts a 12-hour format time string into total minutes since midnight (0 -
1439).
     * Necessary to accurately perform mathematical comparisons on time ranges.
     */
    private int convertToMinutes(String hourString, String amPmIndicator) {
        int hour = Integer.parseInt(hourString);

        // 1 PM to 11 PM becomes 13 to 23
        if (amPmIndicator.equals("PM") && hour != 12) {
            hour += 12;
        }
        // 12 AM becomes 0
        if (amPmIndicator.equals("AM") && hour == 12) {
            hour = 0;
        }

        return hour * 60; // Multiply by 60 to get minutes
    }
}
```

ViewAppointment.java

```
package lepackage;
```

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

/**
 * Secondary GUI class that displays a list of all saved appointments.
 * Allows the user to edit, delete, or mark appointments as done.
 */
public class ViewAppointment extends JFrame {
    private CalendarGUI mainGUI; // Reference to main window to access data and
refresh visuals
    private JTable listTable;
    private DefaultTableModel listModel;

    /**
     * Constructor: Initializes the management window.
     * @param mainGUI The parent CalendarGUI instance
     */
    public ViewAppointment(CalendarGUI mainGUI) {
        this.mainGUI = mainGUI;

        setTitle("View Appointments");
        setSize(750, 400);
        setLayout(new BorderLayout());
        setLocationRelativeTo(mainGUI); // Center over the main window

        initializeTable();
        initializeButtons();
    }

    /**
     * Sets up the list table showing all appointments.
     */
```

```
private void initializeTable() {
    String[] columns = {"Date", "Title", "Category", "Location", "Time", "Status"};

    // Create uneditable model for the list
    listModel = new DefaultTableModel(columns, 0) {
        public boolean isCellEditable(int row, int column) {
            return false;
        }
    };

    listTable = new JTable(listModel);

    // Fetch data from CalendarGUI and populate rows
    populateManageTable();
    add(new JScrollPane(listTable), BorderLayout.CENTER);
}

/**
 * Sets up the bottom action buttons.
 */
private void initializeButtons() {
    JPanel buttonPanel = new JPanel();
    JButton editButton = new JButton("Edit");
    JButton markDoneButton = new JButton("Mark as Done");
    JButton deleteButton = new JButton("Delete");

    // Attach logic to buttons
    editButton.addActionListener(event -> editAppointment());
    markDoneButton.addActionListener(event -> markAppointmentDone());
    deleteButton.addActionListener(event -> deleteAppointment());

    buttonPanel.add(editButton);
    buttonPanel.add(markDoneButton);
    buttonPanel.add(deleteButton);
```

```
        add(buttonPanel, BorderLayout.SOUTH);
    }

    /**
     * Reads from the main GUI's appointment map and adds rows to the JTable.
     */
    private void populateManageTable() {
        listModel.setRowCount(0); // Clear current rows before populating

        // Loop through all dates
        for (String dateKey : mainGUI.getAppointments().keySet()) {
            // Loop through all appointments on each date
            for (CalendarGUI.Appointment appointment :
mainGUI.getAppointments().get(dateKey)) {
                String timeRange = appointment.startTime + " - " + appointment.endTime;

                listModel.addRow(new Object[]{
                    dateKey,
                    appointment.title,
                    appointment.category,
                    appointment.location,
                    timeRange,
                    appointment.status
                });
            }
        }
    }

    /**
     * Changes the status of the selected appointment to "Done".
     */
    private void markAppointmentDone() {
        int selectedRow = listTable.getSelectedRow();
```

```
        if (selectedRow != -1) {
            // Get identifiers from the selected row
            String dateKey = (String) listModel.getValueAt(selectedRow, 0);
            String title = (String) listModel.getValueAt(selectedRow, 1);

            // Find the object in the main map and update it
            for (CalendarGUI.Appointment appointment :
mainGUI.getAppointments().get(dateKey)) {
                if (appointment.title.equals(title)) {
                    appointment.status = "Done";
                }
            }

            // Update the local list UI
            listModel.setValueAt("Done", selectedRow, 5);

            // Tell the main calendar to update (changes cell color from blue to green)
            mainGUI.updateCalendarDisplay();
        } else {
            JOptionPane.showMessageDialog(this, "Please select an appointment to
mark as done.", "Warning", JOptionPane.WARNING_MESSAGE);
        }
    }

    /**
     * Removes the selected appointment from memory and the UI.
     */
    private void deleteAppointment() {
        int selectedRow = listTable.getSelectedRow();
        if (selectedRow != -1) {
            String dateKey = (String) listModel.getValueAt(selectedRow, 0);
            String title = (String) listModel.getValueAt(selectedRow, 1);

            // Remove from main data structure based on title match
```

```
            mainGUI.getAppointments().get(dateKey).removeIf(appointment ->
appointment.title.equals(title));

            // Clean up empty date keys to prevent memory leaks/bugs
            if (mainGUI.getAppointments().get(dateKey).isEmpty()) {
                mainGUI.getAppointments().remove(dateKey);
            }

            // Update UIs
            listModel.removeRow(selectedRow);
            mainGUI.updateCalendarDisplay();
        } else {
            JOptionPane.showMessageDialog(this, "Please select an appointment to
delete.", "Warning", JOptionPane.WARNING_MESSAGE);
        }
    }

    /**
     * Locates the selected appointment and hands it off to AddAppointment for
editing.
     */
    private void editAppointment() {
        int selectedRow = listTable.getSelectedRow();
        if (selectedRow != -1) {
            String dateKey = (String) listModel.getValueAt(selectedRow, 0);
            String title = (String) listModel.getValueAt(selectedRow, 1);

            // Find the exact object to edit
            CalendarGUI.Appointment existingAppt = null;
            for (CalendarGUI.Appointment appt :
mainGUI.getAppointments().get(dateKey)) {
                if (appt.title.equals(title)) {
                    existingAppt = appt;
                    break;
```

```
            }
        }

        if (existingAppt != null) {
            // Open the separated AddAppointment controller in "Edit" mode
            AddAppointment editDialog = new AddAppointment(mainGUI, dateKey,
existingAppt);
            boolean updated = editDialog.showDialog();

            // If the user saved changes, refresh the table
            if (updated) {
                populateManageTable();
            }
        }
    } else {
        JOptionPane.showMessageDialog(this, "Please select an appointment to
edit.", "Warning", JOptionPane.WARNING_MESSAGE);
    }
  }
}
```

Test Case

Test Case ID	Input	Expected Output	Actual Output	Pass/Fail
TC-01	User inputs based on the flow stated in the how to use section	All outputs, warnings, and error must display	All outputs, warnings, and error displays	Pass

TC-02	User inputs nothing in the title text field of add appointment	An error message must display, warning the user about it	An error message displays, warning the user about it	Pass
TC-03	User clicks same starting and time of the appointment	An error message must display, warning the user about it	An error message displays, warning the user about it	Pass
TC-04	User creates appointment that conflicts with the time set of already saved appointment	An error message must display, warning the user about it	An error message displays, warning the user about it	Pass
TC-05	User clicks no appointment as instructed in the view appointment section for editing, marking as done, and deletion	An error message must display, warning the user about it	An error message displays, warning the user about it	Pass
TC-06	User clicks “X” of the program	The program saves the progress into a text file	The program loads the progress from a text file	Pass

Custom Exceptions Explanation

1. **InvalidTimeSequenceException**

   This exception represents a condition where the user attempts to schedule an appointment with a conflicting time (i.e., the End Time occurs before or at the exact same time as the Start Time).

   It is thrown inside the showAppointmentDialog() method during the validation phase. It is caught by a try-catch block within the same method, which suppresses the save action and displays an error dialog to the user.

```
// Throwing the exception

if (endInMinutes <= startInMinutes) {

   throw new InvalidTimeSequenceException("End time must be after the start time!");

}



// Catching the exception

} catch (InvalidTimeSequenceException | AppointmentConflictException ex) {

   JOptionPane.showMessageDialog(this, ex.getMessage(), "Scheduling Error", JOptionPane.ERROR_MESSAGE);

   return false;

}
```

2. **AppointmentConflictException**

   This exception represents a condition where a newly added appointment overlaps with a previously scheduled appointment on the same day, preventing double-booking.

Similar to above, it is thrown in showAppointmentDialog() if the hasTimeConflict() helper method evaluates to true. It is caught in the same try-catch block, alerting the user to pick a different time slot.

```
// Throwing the exception
if (hasTimeConflict(dateKey, startInMinutes, endInMinutes, existingAppt)) {
    throw new AppointmentConflictException("This time overlaps with an
existing appointment!");
}
```

Reflection

We faced several challenges during development. We struggled with date logic and calendar layout. We fixed issues with incorrect day placement through testing and also we handled time conflicts. This project helped us to understand how to build a GUI using Java Swing. We learn how to manage data with HashMap and ArrayList. We handle user input and program events. With more time. We would include reminders for upcoming events. We would store data in a database instead of a text file. We would improve the interface. We created the flowchart and worked on the coding. We handled the project in COMLAB4 but also we worked on it at home during our free time in Discord. We used Claude to help make the calendar design easier to build plus the debugging process but the logic of the code is purely of our own. We also did research to complete the functions on it.